

WEB3 SECURITY RESEARCH / PORTFOLIO CASE STUDY

stNXM by EaseDeFi

Reentrancy in Multi-Protocol Reward Collection

A cross-contract reentrancy analysis of reward collection through owner-configurable staking pools.

Issue #254	20 Nov 2025	Foundry PoC
Sherlock contest	Submission date	Reentrancy path

Researcher: Firman Siregar
Repository target: [sherlock-audit/2025-11-stnxm-by-easedefi](#)
Scope: stNXM reward collection and staking-pool interaction

Portfolio note. This is a standalone presentation of an independent security submission. It is not a client engagement report. Technical claims are based on the submitted report and PoC material supplied for this case study.

1. Executive Summary

The reported vulnerability concerns `stNXM.getRewards()`, which iterates across registered staking positions and invokes an external `pool.withdraw()` call through `_withdrawFromPool()`. The report argues that staking pools are treated as trusted even though the pool address is stored through an owner-controlled staking configuration path and the external call is made without an explicit reentrancy guard.

The core security concern is a cross-contract interaction boundary: while reward collection is in progress, a malicious staking-pool implementation can call back into `getRewards()` before the outer execution has completed. The submitted report identifies this as a violation of the checks-effects-interactions pattern and a missing reentrancy defense.

The submitted impact was repeated reward claims, state corruption, and potentially fund theft if reentrant execution can manipulate reward accounting or withdrawals. The supplied PoC demonstrates the malicious pool and reentrant callback path; the material supplied for this portfolio does **not** establish a concrete drained-balance result. The fund-loss impact is therefore presented as the reported potential impact, not as a measured PoC outcome.

Finding at a glance

Target	stNXM by EaseDeFi
Issue	#254 — Reentrancy in Multi-Protocol Reward Collection
Submitted	20 November 2025, 2:47:26 PM GMT+7
Primary surface	<code>stNXM.getRewards()</code> → <code>_withdrawFromPool()</code> → <code>pool.withdraw()</code>
Attack primitive	Malicious staking-pool callback into <code>getRewards()</code>
Privilege assumption	Owner must configure the malicious pool
PoC framework	Foundry / Solidity 0.8.26
Classification	Cross-contract reentrancy / reward-accounting risk

2. Finding at a Glance

The security model depends on an assumption that configured staking pools are benign. The report challenges that assumption by treating the staking pool as an external contract boundary capable of executing arbitrary code during withdrawal.

Control flow

```
getRewards()
|
+-- iterate tokenIds[]
|
+-- _withdrawFromPool(pool, tokenId, ...)
    |
    +-- pool.withdraw(...)
        |
        +-- malicious pool calls back
            |
            +-- stNXM.getRewards()
```

Reported root causes

- No explicit reentrancy guard on reward collection despite the external call.
- External staking-pool interaction occurs before the overall reward-collection operation has completed.
- The design assumes staking pools are trusted rather than treating pool implementations as an untrusted execution boundary.
- The report identifies state updates after external interaction as a checks-effects-interactions violation.

Important precondition

The submitted attack model requires the owner to configure a malicious staking-pool address in the **tokenIdToPool** mapping through **stakeNxM()**. This makes the issue different from an arbitrary permissionless reentrancy bug: exploitation depends on a malicious, compromised, or incorrectly accepted pool being configured.

3. Technical Root Cause

The submitted report points to **getRewards()** as the vulnerable function. Its loop walks every registered token ID and delegates reward withdrawal to the corresponding staking pool.

```
function getRewards() external update returns (uint256 rewards) {
    for (uint256 i = 0; i < tokenIds.length; i++) {
        uint256 tokenId = tokenIds[i];
        rewards += _withdrawFromPool(
            tokenIdToPool[tokenId],
            tokenId,
            false,
            true,
            tokenIdToTranches[tokenId]
        );
    }
    // ... additional logic
}
```

The important property is that **_withdrawFromPool()** reaches an external staking-pool contract. Its **withdraw()** implementation can execute arbitrary code and return control to stNXM before the outer reward-collection operation has finished.

Security invariant

Reward collection should not be recursively executable across the same stNXM state transition. Each logical reward-collection operation should consume the relevant staking state exactly once and should not allow an external pool callback to start a second collection against partially updated state.

4. Exploit Mechanics

The attack is a two-contract interaction: stNXM calls a staking pool to withdraw rewards, and the malicious pool uses that call as an opportunity to regain control of stNXM.

1. Pool configuration — A malicious staking-pool address is placed into the stNXM pool mapping through the owner-controlled staking path.

2. Reward accumulation — The protocol has reward state available for collection.

3. Trigger — A caller invokes `getRewards()`.

4. External call — stNXM reaches the malicious pool's `withdraw()` function.

5. Callback — The pool calls `stNXM.getRewards()` before the outer reward collection has completed.

6. Nested execution — The nested call reprocesses the reward-collection path against intermediate state.

7. Potential consequence — Depending on accounting and state-transition ordering, repeated reward claims, corrupted accounting, or fund extraction may become possible.

The submitted report describes the attack as potentially continuing until gas limits are reached or the contract is drained. That upper-bound impact is part of the report's threat model; the supplied PoC excerpt does not provide a measured full-drain execution.

5. Proof of Concept

The PoC introduces a **ReentrantStakingPool** implementing **IStakingPool**. Its `withdraw` function conditionally calls back into `stNxm.getRewards()`, with a reentrancy counter limiting recursion during the test.

```
function withdraw(
  uint256,
  bool,
  bool,
  uint256[] memory
) external override returns (uint256, uint256) {
  if (shouldReenter && reentrancyCount < 10) {
    reentrancyCount++;
    stNxm.getRewards();
  }
  return (0, 0);
}
```

This isolates the trust-boundary failure: the staking pool does not need a sophisticated reward calculation. It only needs to prove that stNXM permits external pool code to regain control and call the reward-collection entry point.

The PoC also includes extreme-input and repeated-call tests. The zero-value test exercises malicious-pool configuration with empty values; the max-value test checks behavior around uint256 extremes; and the repeated-reentrancy test repeatedly invokes the reward path while recording the malicious pool's callback count.

6. PoC Test Design

The supplied test suite contains three relevant scenarios.

Test	Purpose	Evidence represented
test_ReentrancyWithZeroValues()	Exercise malicious-pool configuration with zero/empty inputs.	Reentrancy counter assertion / edge-case path.
test_ReentrancyWithMaxUint256()	Exercise extreme uint256 inputs and expect failure.	Arithmetic/revert behavior around maximum values.
test_RepeatedReentrancyCalls()	Repeatedly invoke getRewards() and allow pool callbacks.	Records reentrancy attempts via reentrancyCount.

Interpretation

The PoC is best presented as a **reentrancy-path demonstration**. It establishes the malicious pool and callback mechanism and tests repeated callback behavior. It does not, in the supplied material, demonstrate an actual duplicated reward amount, corrupted state variable, or specific stolen-balance delta.

For a portfolio, this distinction is useful: it demonstrates disciplined security reporting rather than treating a plausible impact statement as equivalent to a proven economic drain.

7. Security Impact Analysis

The submitted report identifies three potential impact classes.

- 1. Repeated reward claims.** A nested `getRewards()` execution could process reward-collection logic more than once during a single outer operation if accounting is not protected.
- 2. State corruption.** Recursive execution can observe partially updated state. Depending on the exact ordering of accounting operations, this can produce inconsistent reward or staking state.
- 3. Fund theft.** The report states that an attacker could ultimately drain the contract balance and that stakers could suffer complete fund loss. The supplied PoC does not quantify or demonstrate this final outcome, so it should be described as the reported potential impact rather than a measured result.

Threat-model boundary

Because the malicious pool must first be accepted into `tokenIdToPool` through the owner-controlled staking flow, exploitation depends on the system accepting a malicious or compromised pool. Pool onboarding/configuration is therefore a critical security boundary.

Why this matters

A protocol that aggregates external pools effectively extends its trusted computing base beyond stNXM itself. Each integration can become capable of influencing execution at sensitive points in the accounting lifecycle.

8. Recommended Remediation

The submitted mitigation recommends several layers of defense.

Primary defense — reentrancy protection. Add a robust nonReentrant mechanism to reward collection and related state-changing entry points that can reach the same accounting state.

```
modifier nonReentrant() {
    require(!_reentrancyLock, "Reentrancy detected");
    _reentrancyLock = true;
    _;
    _reentrancyLock = false;
}
```

State-ordering defense. Apply checks-effects-interactions consistently. State that must be consumed exactly once should be updated before calling untrusted external contracts where practical.

Pool trust boundary. Whitelist staking pools and subject pool implementations to security review before they can participate in reward collection.

Circuit breaker. Consider the ability to pause or isolate interaction with a problematic pool without disabling unrelated protocol functionality.

Regression tests. Include malicious-pool test doubles that re-enter through every externally reachable callback path, not only withdraw().

Defense in depth

Even if staking pools are intended to be trusted, reentrancy protection remains valuable because trust assumptions can fail through upgrades, misconfiguration, compromised deployments, or future pool implementations.

9. Validation & Research Methodology

The investigation is presented as a contract-level review of the reward-collection control flow followed by an adversarial Foundry test.

Research sequence

- Identify reward collection as a value-sensitive state transition.
- Trace `getRewards()` into `_withdrawFromPool()`.
- Identify `pool.withdraw()` as an external control-flow boundary.
- Challenge the assumption that configured pools are non-malicious.
- Build a malicious `IStakingPool` implementation that calls back into `getRewards()`.
- Exercise zero, extreme, and repeated-reentrancy inputs.
- Evaluate whether the callback can occur before outer reward collection completes.

This methodology fits an economic-invariant-first review: the important question is not simply whether an external call exists, but whether a reward-collection transition remains atomic and single-use when control can return from an external dependency.

Suggested regression invariant

For a single logical reward-collection operation:

```
reward state consumed once
+ pool withdrawal executed once
+ no recursive collection against intermediate state
= atomic reward settlement
```

10. Portfolio Assessment

This case study demonstrates **trust-boundary analysis across protocol components**, rather than only pattern-based Solidity review.

It shows capability in:

- External-call control flow and callback surfaces.
- Reentrancy beyond classic token-transfer patterns.
- Protocol assumptions around configurable external contracts.
- Reward accounting and state-transition atomicity.
- Adversarial Foundry testing with malicious contract doubles.
- Separating a demonstrated primitive from a claimed downstream economic impact.

Recommended GitHub project positioning

stNXM — Reentrancy in Multi-Protocol Reward Collection

A security research case study analyzing how an externally configurable staking-pool implementation can reenter stNXM during multi-protocol reward collection. Includes a Foundry malicious-pool PoC covering recursive `getRewards()` calls, edge-case inputs, and repeated reentrancy attempts.

Suggested tags

Smart Contract Security · Solidity · DeFi · Reentrancy · Reward Accounting · Cross-Contract Calls · Foundry · Threat Modeling

11. Submission Record

The following details are preserved from the submitted material.

Platform	Sherlock
Contest	stNXM by EaseDeFi
Repository	sherlock-audit/2025-11-stnxm-by-easedefi
Issue	#254
Title	Reentrancy in Multi-Protocol Reward Collection
Submitted	20 November 2025 at 2:47:26 PM GMT+7
Reported severity	Not explicitly stated in supplied submission
Triage outcome	Not provided in supplied material

Portfolio disclosure note

This was a security-research submission to a Sherlock contest, not a commissioned client engagement. Present it as independent research rather than client audit work.

12. Researcher's Notes

The strongest lesson from this finding is that **external protocol components are part of the attack surface**. A staking pool may be conceptually treated as infrastructure, but from the EVM's perspective its `withdraw()` function is executable code with the ability to call back into its caller.

The security review therefore moves beyond the simplistic question — “Does this function have a reentrancy guard?” — toward the more useful question:

“What assumptions about external components must remain true for this economic state transition to be atomic?”

That framing is especially relevant to DeFi systems that aggregate multiple protocols, pools, adapters, strategies, or reward sources. Each integration expands the set of contracts capable of influencing execution at sensitive points in the accounting lifecycle.

Audit. Break. Repeat.

Economic invariants first. Code second. Proof third.

Firman Siregar · Web3 Security Researcher · Smart Contracts & DeFi